Tasks on k8s-02

1. **Create a Simple Pod Using YAML Task: Write a YAML file to create a Pod named firstpod with an nginx container. Verify the Pod creation using kubectl get pods and check the logs of the container using kubectl logs firstpod.**

Objective:

Create a Pod named firstpod with an Nginx container and verify its status and logs.

Implementation Steps:

1. Connect to the Kubernetes master node using SSH.
2. Verify the cluster nodes.

   kubectl get nodes
3. Create a YAML file.

   vi firstpod.yml
4. Add the following configuration:

   apiVersion: v1

   kind: Pod

   metadata:

    name: firstpod

   spec:

    containers:

     - name: nginx

       image: nginx
5. Save and exit the file.
6. Create the Pod using the YAML file.

   kubectl create -f firstpod.yml
7. Verify the Pod status.

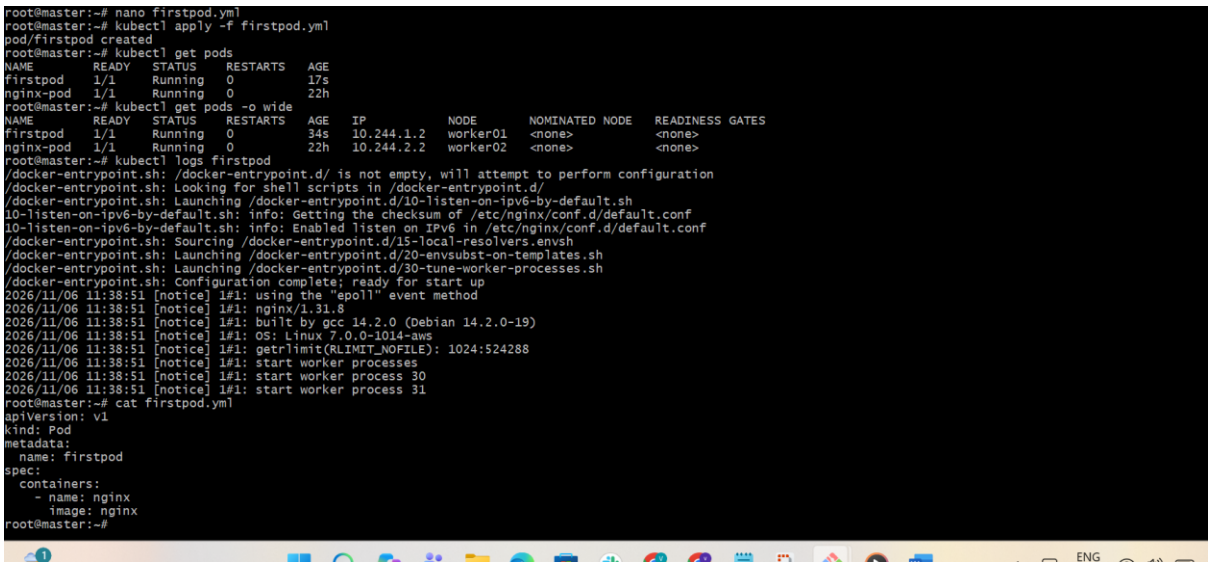
   kubectl get pods
8. Check the detailed information of the Pod.

   kubectl describe pod firstpod

9. Check the container logs.

   kubectl logs firstpod

   Expected Result: The firstpod Pod should be created successfully with an Nginx container.



2. Set Environment Variables in a Pod Task: Modify the YAML file to include environment variables myname: sabair and City: Hyderabad. Deploy the Pod and use kubectl exec <pod_name> -- env to check if the environment variables are set properly.

Objective:

Configure environment variables inside the Nginx container and verify them using the kubectl exec command.

Implementation Steps:

1. Open the existing YAML file.

   vi firstpod.yml

2. Modify the configuration by adding environment variables under the container specification.

   apiVersion: v1

   kind: Pod

   metadata:

   name: firstpod

   spec:

```
containers:
 - name: nginx
   image: nginx
   env:
    - name: myname
      value: "sabair"
    - name: City
      value: "Hyderabad"
```

3. Save and exit the file.
4. Apply the updated configuration.

   kubectl apply -f firstpod.yml
5. Verify the Pod status.

   kubectl get pods
6. Check the environment variables inside the container.

   kubectl exec firstpod -- env
7. Verify the specific environment variables.
8. kubectl exec firstpod -- printenv myname

   kubectl exec firstpod -- printenv City

   Expected Result: The output should display:



```
root@master:~# kubectl delete pod firstpod
kubectl apply -f firstpod.yml
pod "firstpod" deleted from default namespace
pod/firstpod created
root@master:~# kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
firstpod    1/1     Running   0          29s
nginx-pod   1/1     Running   0          22h
root@master:~# kubectl exec firstpod -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=firstpod
NGINX_VERSION=1.31.8
NJS_VERSION=1.0.2
NJS_RELEASE=1~trixie
ACME_VERSION=0.4.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
myname=sabair
City=Hyderabad
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
HOME=/root
root@master:~#
```

3. Deploy a Pod with Commands (Args) in YAML Task: Modify the YAML file to add args that instruct the container to sleep for 50 seconds. Deploy the Pod and use kubectl describe pod to verify the args are correctly passed to the container.

Objective:

Configure the container to execute a sleep argument for 50 seconds.

Implementation Steps:

1. Open the existing YAML file.

   vi firstpod.yml

2. Add the args field under the container specification.

   apiVersion: v1

   kind: Pod

   metadata:

    name: firstpod

   spec:

    containers:

     - name: nginx

       image: nginx

       args:

- sleep

- "50"

3. Save and exit the file.
4. Delete the existing Pod because changing container arguments requires recreating the Pod.

   kubectl delete pod firstpod

5. Create the Pod again.

   kubectl apply -f firstpod.yml

6. Verify the Pod status.

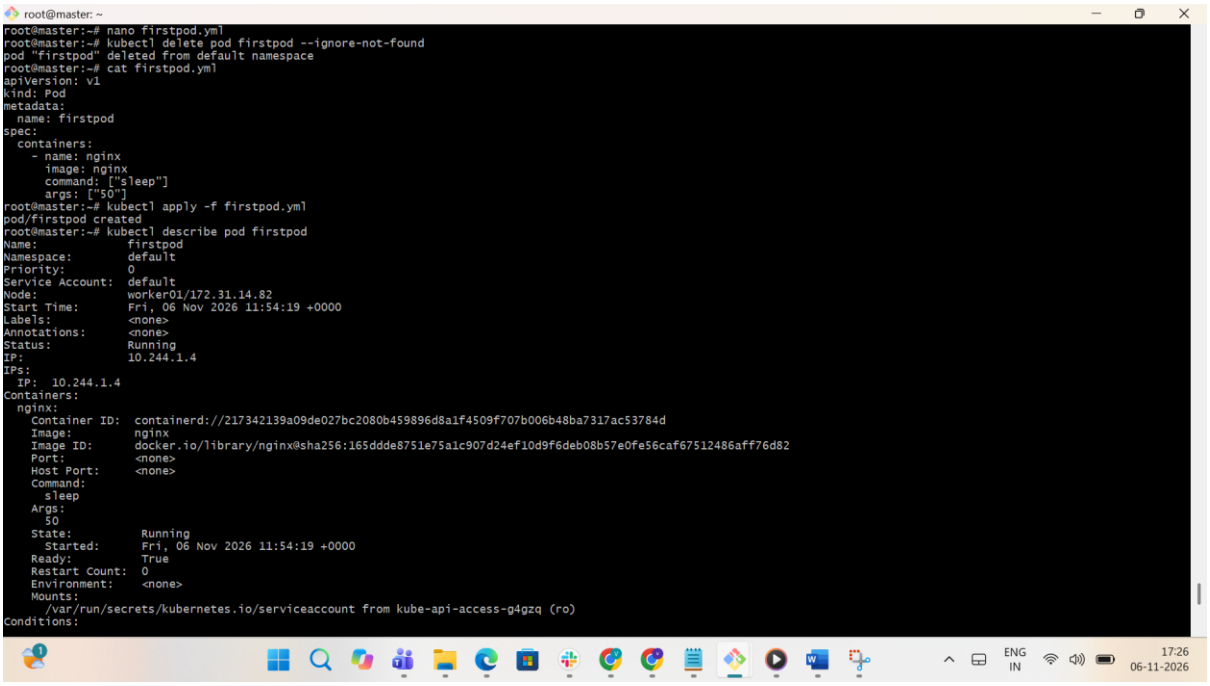
   kubectl get pods

7. Check the container configuration.

   kubectl describe pod firstpod

8. Verify the command and arguments under the container details.

   Expected Result: The container receives the sleep 50 arguments. Since Nginx does not normally provide a sleep executable as its default entrypoint, use command: ["sleep"] with args: ["50"] if the intention is to run a standalone sleep container. In that case, the container will sleep for 50 seconds and then exit.



```
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-g4gzq:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  25s   default-scheduler  Successfully assigned default/firstpod to worker01
  Normal  Pulling    25s   kubelet            spec.containers{nginx}: Pulling image "nginx"
  Normal  Pulled     25s   kubelet            spec.containers{nginx}: Successfully pulled image "nginx" in 143ms (143ms including waiting). Image size
  Normal  Created    25s   kubelet            spec.containers{nginx}: Container created
  Normal  Started    25s   kubelet            spec.containers{nginx}: Container started
root@master:~# kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
firstpod    1/1     Running   0          47s
nginx-pod   1/1     Running   0          22h
root@master:~# kubectl get pods
NAME        READY   STATUS    RESTARTS     AGE
firstpod    1/1     Running   1 (9s ago)   59s
nginx-pod   1/1     Running   0            22h
root@master:~# kubectl delete pod firstpod --ignore-not-found
pod "firstpod" deleted from default namespace
root@master:~#
```

4. Create a Pod with Two Containers Task: Create a YAML file to define a Pod with two nginx containers inside. Use kubectl exec to access both containers and verify that both containers can communicate through the same network (e.g., using telnet between them).

Objective:

Create a single Pod containing two Nginx containers and verify that both containers share the same network.

Implementation Steps:

1. Create a YAML file.

   vi multicontainer.yml

2. Add the following configuration:

   apiVersion: v1

   kind: Pod

   metadata:

    name: multicontainer-pod

   spec:

    containers:

     - name: nginx1

       image: nginx

     - name: nginx2

image: nginx

3. Save and exit the file.
4. Deploy the Pod.

   kubectl apply -f multicontainer.yml
5. Verify both containers are running.

   kubectl get pods

   The READY column should display 2/2.
6. Describe the Pod to verify both container details.

   kubectl describe pod multicontainer-pod
7. Access the first container.

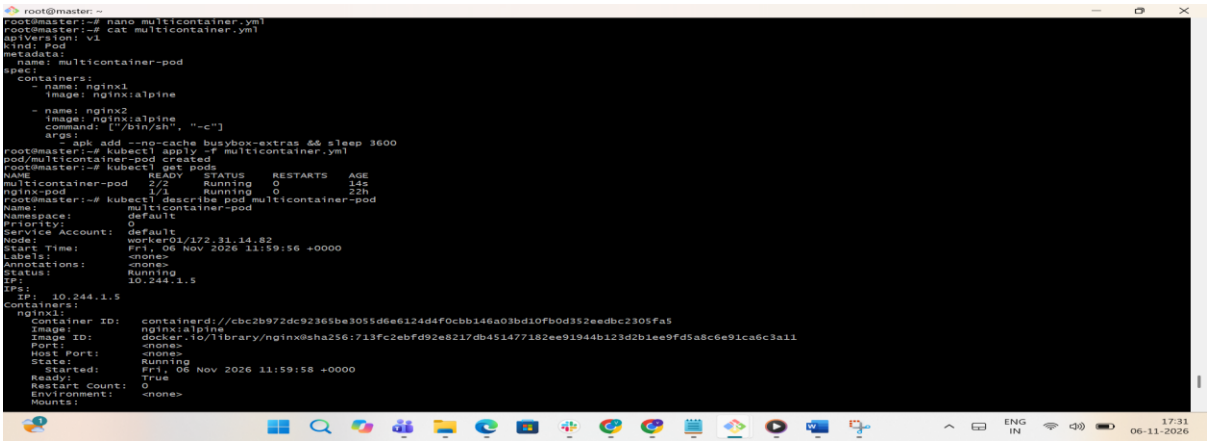
   kubectl exec -it multicontainer-pod -c nginx1 -- /bin/bash
8. Access the second container.

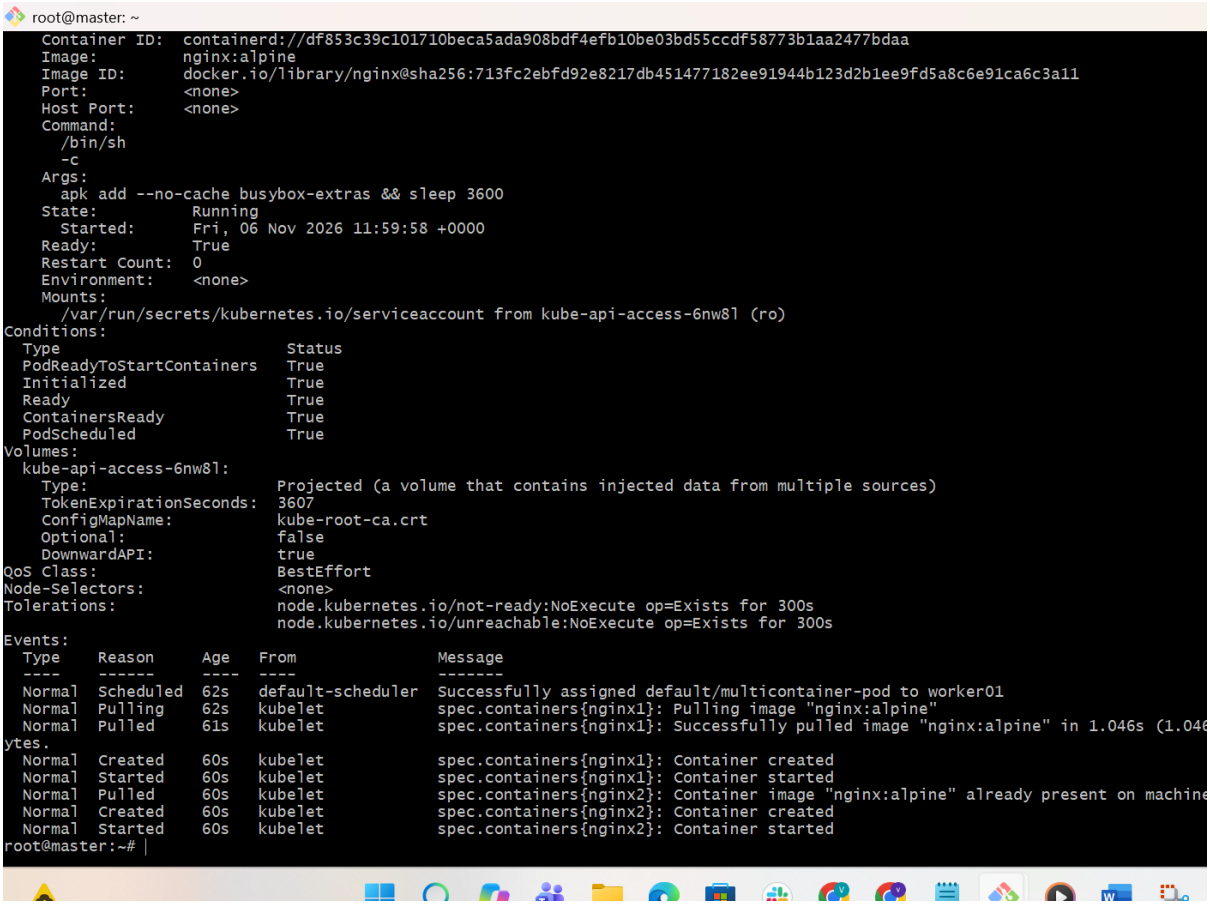
   kubectl exec -it multicontainer-pod -c nginx2 -- /bin/bash
9. Verify the Nginx server is listening on port 80.

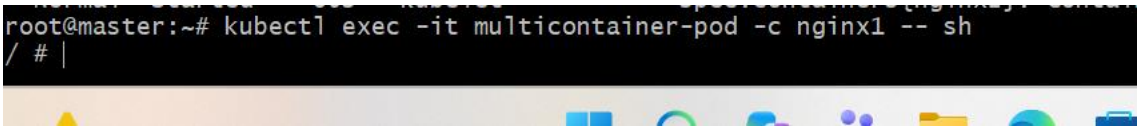
   curl localhost
10. To verify communication between the containers, execute a request to the shared Pod IP on port 80 from the other container.

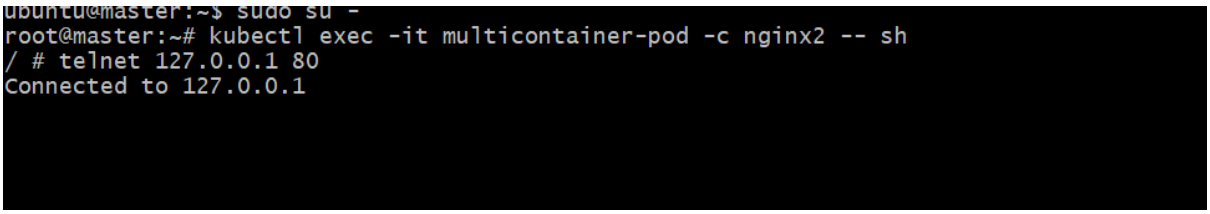
    kubectl exec multicontainer-pod -c nginx1 -- curl -I http://localhost:80

    Expected Result: Both containers should run inside the same Pod and share the same network namespace and Pod IP. They can communicate through localhost, but they cannot both independently bind to the same port on the shared network namespace.









5. **Set Up an Init Container in a Pod Task: Modify the YAML to include an init container that sleeps for 30 seconds before the main containers start. Verify the init container's execution using kubectl describe pod and check the logs to confirm its completion.**

Objective:

Configure an Init Container to sleep for 30 seconds before the main application container starts.

Implementation Steps:

1. Create a YAML file.

   vi initpod.yml

2. Add the following configuration:

```
apiVersion: v1
kind: Pod
metadata:
  name: initpod
spec:
  initContainers:
    - name: init-container
      image: busybox
      command: ["sh", "-c", "sleep 30"]

  containers:
    - name: nginx
      image: nginx
```

3. Save and exit the file.
4. Deploy the Pod.

   kubectl apply -f initpod.yml
5. Check the Pod status.

   kubectl get pods
6. Observe the Pod while the Init Container is executing.

   kubectl get pods -w
7. Describe the Pod to check the Init Container details.

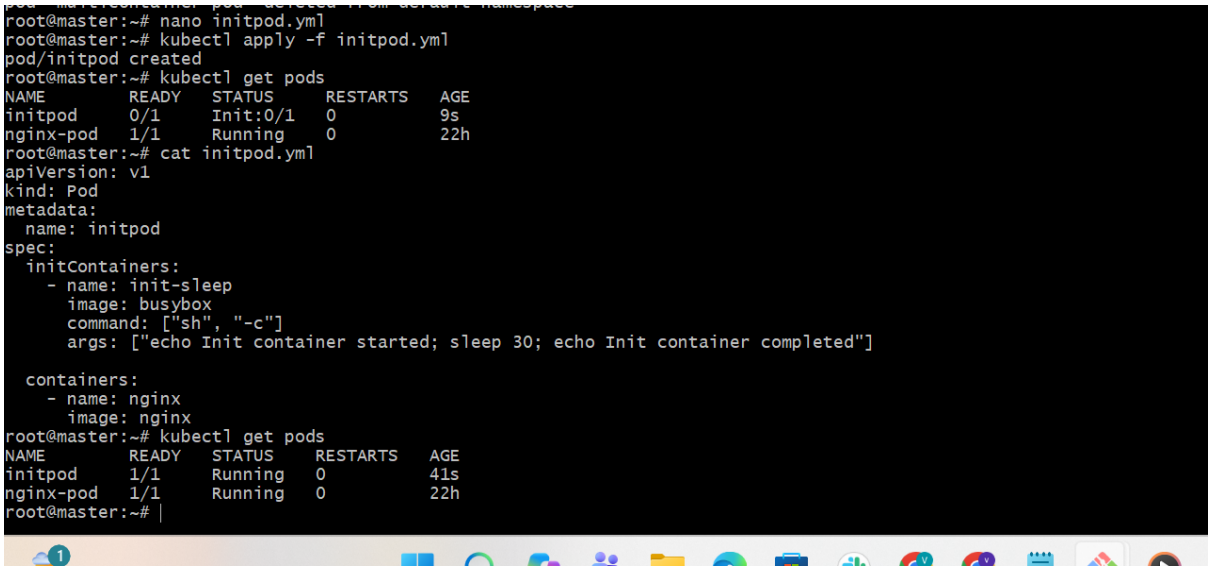
   kubectl describe pod initpod
8. Check the Init Container logs.

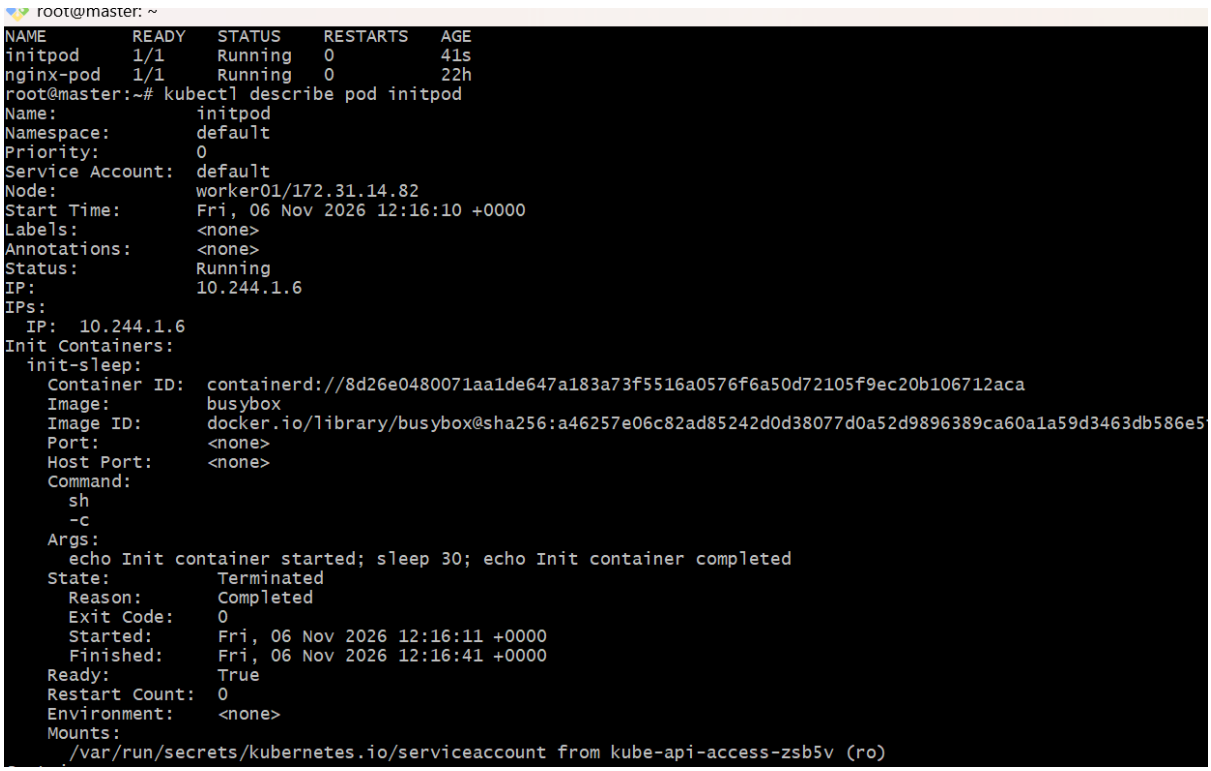
   kubectl logs initpod -c init-container
9. Verify the Init Container completion status.

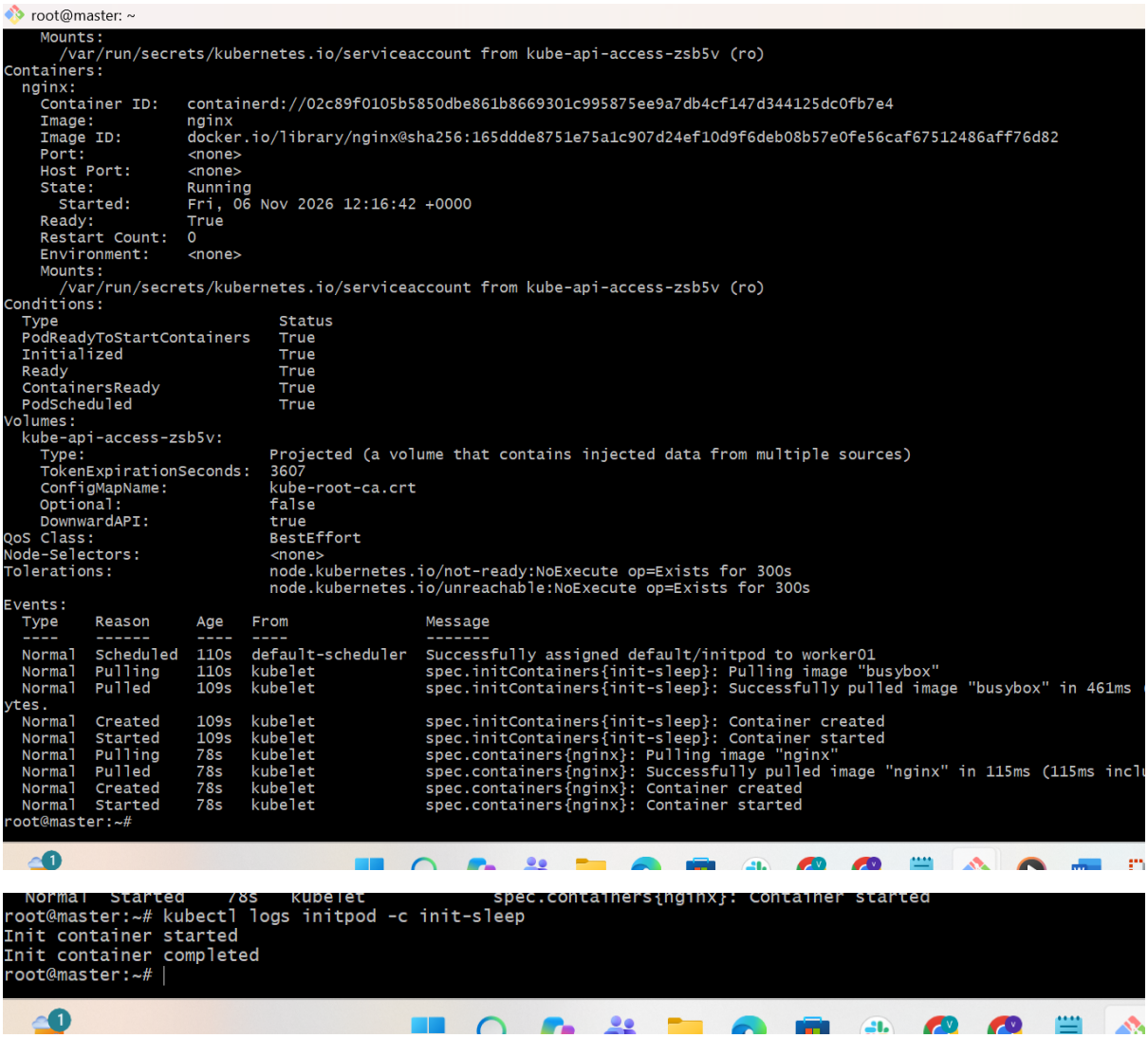
   kubectl get pod initpod
10. Verify that the main Nginx container has started successfully.

    kubectl exec initpod -- nginx -v

Expected Result: The Init Container should execute the sleep command for 30 seconds. After successful completion, the main Nginx container should start. The Init Container status should show Completed.







6. **Run a Dry Run Command to Generate YAML Task: Use the kubectl run nginx --image=nginx --dry-run=client -o yaml command to generate a Pod YAML definition. Modify the generated YAML to suit specific requirements (e.g., labels or environment variables) and deploy it.**

   Implementation steps:

Objective:

Generate a Kubernetes Pod YAML file automatically using the kubectl dry-run command and modify it according to the requirements.

Implementation Steps:

1. Execute the following command to generate an Nginx Pod YAML definition.

   kubectl run nginx --image=nginx --dry-run=client -o yaml

2. To save the generated YAML directly into a file, execute:

   kubectl run nginx --image=nginx --dry-run=client -o yaml > nginx.yml

3. Open the generated YAML file.

   vi nginx.yml

4. Modify the YAML file and add labels and environment variables.

```
apiVersion: v1
kind: Pod
metadata:
 name: nginx
 labels:
  environment: test
  app: nginx
spec:
 containers:
  - name: nginx
    image: nginx
    env:
     - name: City
       value: "Hyderabad"
```

5. Save and exit the file.
6. Deploy the Pod.

   kubectl apply -f nginx.yml

7. Verify the Pod.

   kubectl get pods

8. Verify the labels.

   kubectl get pods --show-labels

9. Check the environment variable.

   kubectl exec nginx -- printenv City

   Expected Result: The YAML file should be generated successfully, modified with the required labels and environment variables, and deployed into the Kubernetes cluster.

```
root@master: ~
root@master:~# kubectl run nginx --image=nginx --dry-run=client -o yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: nginx
  name: nginx
spec:
  containers:
  - image: nginx
    name: nginx
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
root@master:~# kubectl run nginx --image=nginx --dry-run=client -o yaml > nginx.yml
root@master:~# vi nginx.yml
root@master:~# nano nginx.yml
root@master:~# cat nginx.yml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    environment: test
spec:
  containers:
    - name: nginx
      image: nginx
      env:
        - name: City
          value: "Hyderabad"
root@master:~# kubectl apply -f nginx.yml
pod/nginx created
```

```
root@master:~# kubectl apply -f nginx.yml
pod/nginx created
root@master:~# kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
nginx       1/1     Running   0          10s
nginx-pod   1/1     Running   0          22h
root@master:~# kubectl get pods --show-labels
NAME        READY   STATUS    RESTARTS   AGE   LABELS
nginx       1/1     Running   0          23s   environment=test
nginx-pod   1/1     Running   0          22h   run=nginx-pod
root@master:~# kubectl exec nginx -- env
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=nginx
NGINX_VERSION=1.31.8
NJS_VERSION=1.0.2
NJS_RELEASE=1~trixie
ACME_VERSION=0.4.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
City=Hyderabad
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
HOME=/root
```

7. **Use kubectl apply vs kubectl create Task: Create a YAML file to define a Pod. First, deploy it using kubectl create -f <file_name>.yml and then modify the YAML (e.g., change the image version). Use kubectl apply to redeploy and verify the difference between both commands.**

Objective:

Understand the difference between kubectl create and kubectl apply by creating and updating a Pod using YAML files.

Implementation Steps:

1. Create a YAML file.

   vi createpod.yml

2. Add the following configuration:

   apiVersion: v1

   kind: Pod

   metadata:

    name: testpod

   spec:

    containers:

     - name: nginx

       image: nginx:latest

3. Save and exit the file.
4. Create the Pod using the kubectl create command.

   kubectl create -f createpod.yml

5. Verify the Pod.

   kubectl get pods

6. Now modify the image version in the YAML file.

   vi createpod.yml

7. Change the image from nginx:latest to nginx:1.27.

   image: nginx:1.27

8. Save and exit the file.
9. Execute the create command again.

   kubectl create -f createpod.yml

10. Observe that Kubernetes returns an AlreadyExists error because the Pod already exists.
11. Now execute the apply command.

    kubectl apply -f createpod.yml

    12. Verify the updated image.

    kubectl describe pod testpod

    13. Check the Pod status.

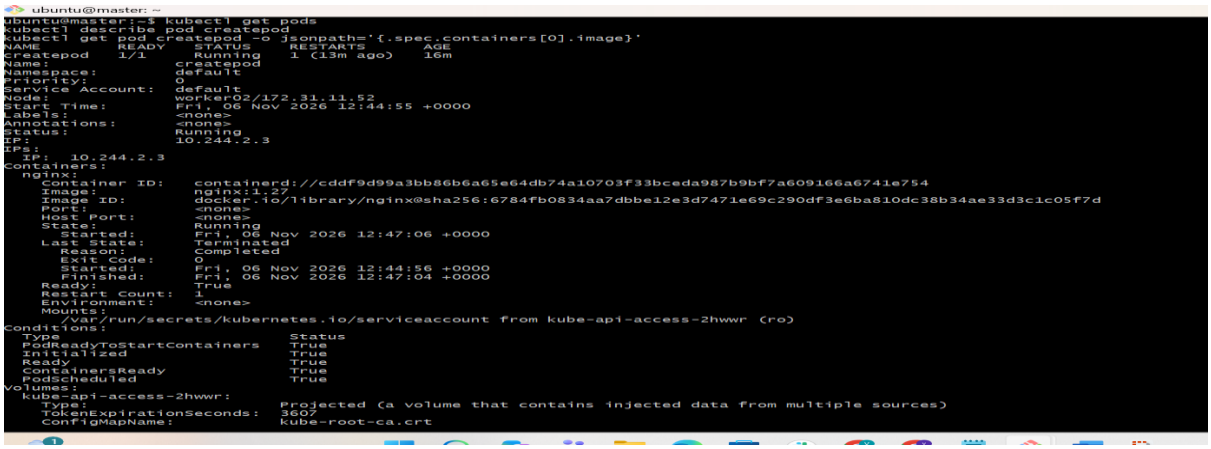
kubectl get pods

Difference Between kubectl create and kubectl apply:

kubectl create	**kubectl apply**
Imperative object creation	Declarative object management
Creates a new resource	Creates or updates a resource
Returns AlreadyExists if the resource already exists	Checks the configuration and applies changes
Does not automatically update an existing resource from the YAML	Maintains the desired configuration using the YAML

Expected Result: The create command creates the Pod initially. When executed again, it returns an AlreadyExists error. The apply command can update the existing Pod configuration based on the YAML file.

```
root@master:~# nano createpod.yml
root@master:~# cat createpod.yml
apiVersion: v1
kind: Pod
metadata:
  name: createpod
spec:
  containers:
    - name: nginx
      image: nginx:latest
root@master:~# kubectl create -f createpod.yml
pod/createpod created
root@master:~# kubectl get pods
NAME         READY   STATUS    RESTARTS   AGE
createpod    1/1     Running   0          39s
root@master:~# nano createpod.yml
root@master:~# cat createpod.yml
apiVersion: v1
kind: Pod
metadata:
  name: createpod
spec:
  containers:
    - name: nginx
      image: nginx:1.27
root@master:~# kubectl apply -f createpod.yml
```



8. Edit an Existing Pod Configuration Task: Use kubectl edit pod <pod_name> to modify the running Pod's environment variables or image. After making the changes, verify if they took effect by checking the container logs or environment variables using kubectl exec.

Objective:

Modify an existing Pod configuration using the kubectl edit command and verify the changes.

Implementation Steps:

1. Check the existing Pods.

   kubectl get pods

2. Select the Pod that needs to be modified.
3. Execute the edit command.

   kubectl edit pod firstpod

4. The Pod configuration will open in the default editor.
5. Locate the container image configuration.
6. containers:
7. - name: nginx

     image: nginx

8. Change the image version.

   image: nginx:1.27

9. Save and exit the editor.
10. Kubernetes will attempt to update the Pod configuration.
11. Verify the updated Pod details.

    kubectl describe pod firstpod

12. Check the container image.

    kubectl get pod firstpod -o jsonpath='{.spec.containers[0].image}'

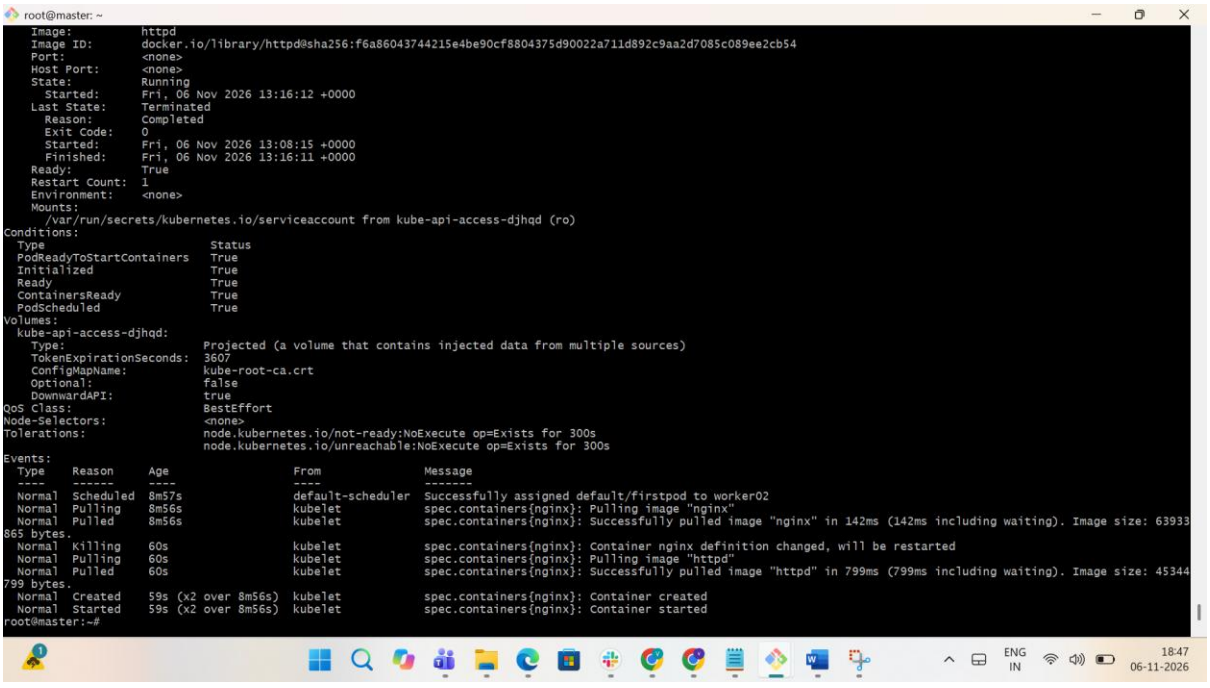
    11. Verify the running container.

    kubectl get pods

    Expected Result: The container image should be updated successfully if the requested change is permitted.

```
ubuntu@master:~$ cat firstpod.yml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
    - name: nginx
      image: nginx
ubuntu@master:~$ kubectl apply -f firstpod.yml
pod/firstpod created
ubuntu@master:~$ kubectl get pods
NAME       READY   STATUS    RESTARTS   AGE
firstpod   1/1     Running   0          10s
```

```
root@master:~# kubectl edit pod firstpod
pod/firstpod edited
root@master:~# cat firstpod.yml
apiVersion: v1
kind: Pod
metadata:
  name: firstpod
spec:
  containers:
    - name: nginx
      image: nginx
      command: ["sleep"]
      args: ["50"]
root@master:~# kubectl get pods
NAME       READY   STATUS    RESTARTS      AGE
firstpod   1/1     Running   1 (46s ago)   8m43s
root@master:~# kubectl describe pod firstpod
Name:             firstpod
Namespace:        default
Priority:         0
Service Account:  default
Node:             worker02/172.31.11.52
Start Time:       Fri, 06 Nov 2026 13:08:14 +0000
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.2.4
IPs:
  IP:  10.244.2.4
Containers:
  nginx:
    Container ID:   containerd://01ee85f80bd943fccd247e12b3606eb34117ce2e1486a7198a3daca3cb75a22d
    Image:          httpd
    Image ID:       docker.io/library/httpd@sha256:f6a86043744215e4be90cf8804375d90022a711d892c9aa2d7085c089ee2cb54
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Fri, 06 Nov 2026 13:16:12 +0000
    Last State:     Terminated
      Reason:       Completed
```

9. Expose a Pod Using a Service Task: Create a YAML file to expose your firstpod using a Service (ClusterIP). Ensure that your service is exposing the Pod on port 80 and verify it using kubectl get svc.

Objective:

Create a ClusterIP Service to expose the firstpod Pod on port 80 and enable communication within the Kubernetes cluster.

Implementation Steps:

1. Verify that the Pod is running.

   kubectl get pods

2. Ensure that the Pod has the required label. Open the Pod YAML file.

   vi firstpod.yml

3. Add a label under metadata.

4. metadata:

5. name: firstpod

6. labels:

      app: firstpod

7. Apply the updated configuration. If the label cannot be changed on the existing Pod, recreate it.

   kubectl delete pod firstpod

   kubectl apply -f firstpod.yml

8. Create a Service YAML file.

   vi firstpod-service.yml

9. Add the following Service configuration.

   apiVersion: v1

   kind: Service

   metadata:

    name: firstpod-service

   spec:

    type: ClusterIP

    selector:

```
    app: firstpod
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

10. Save and exit the file.
11. Create the Service.

    kubectl apply -f firstpod-service.yml

12. Verify the Service.

    kubectl get svc

13. Check the detailed Service configuration.

    kubectl describe svc firstpod-service

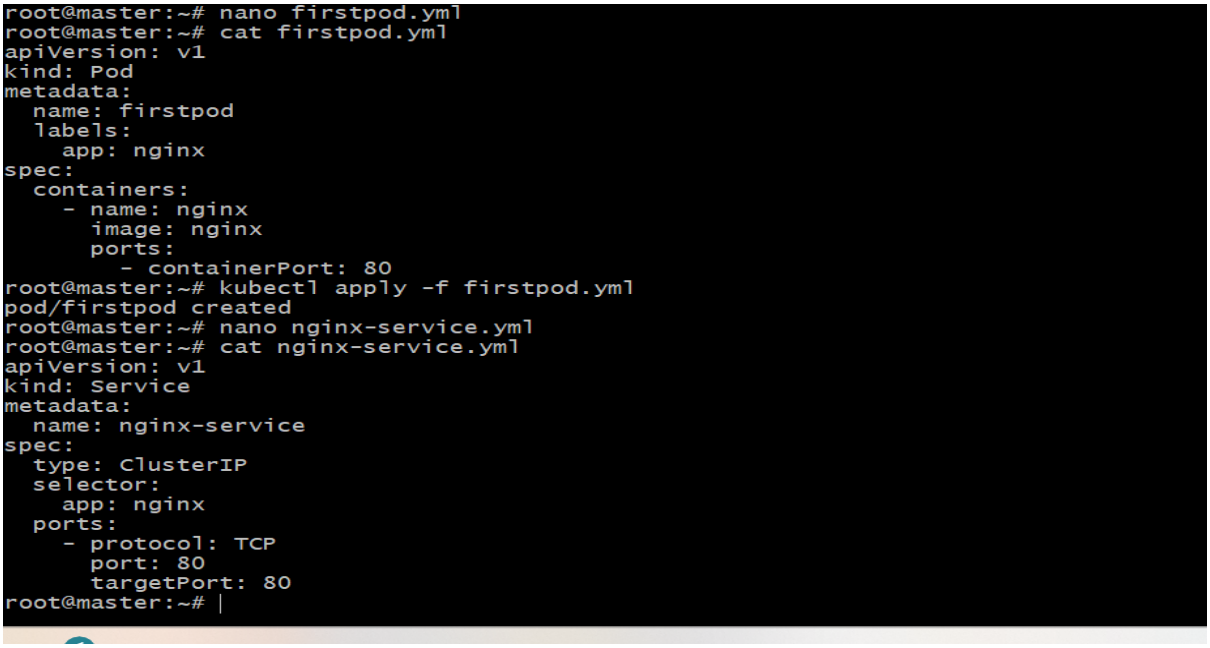
14. Verify that the Service is connected to the Pod.

    kubectl get endpoints firstpod-service

15. Test the Service from inside the cluster.

    kubectl run test-client --image=busybox --restart=Never -it --rm -- wget -qO-http://firstpod-service

    Expected Result: The ClusterIP Service should be created successfully and route requests on port 80 to the Nginx container in firstpod.



```
root@master:~# kubectl apply -f nginx-service.yml
service/nginx-service created
root@master:~# kubectl get svc
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
kubernetes      ClusterIP   10.96.0.1       <none>        443/TCP   24h
nginx-service   ClusterIP   10.100.97.54    <none>        80/TCP    9s
root@master:~# kubectl describe svc nginx-service
Name:                     nginx-service
Namespace:                default
Labels:                   <none>
Annotations:              <none>
Selector:                 app=nginx
Type:                     ClusterIP
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.100.97.54
IPs:                      10.100.97.54
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
Endpoints:                10.244.2.5:80
Session Affinity:         None
Internal Traffic Policy:  Cluster
Events:                   <none>
root@master:~#
```

```
oot@master:~# kubectl get endpoints nginx-service
arning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
AME            ENDPOINTS       AGE
ginx-service   10.244.2.5:80   2m8s
oot@master:~# kubectl run test-pod --image=busybox --restart=Never --rm -it -- wget -qO- http://nginx-service
!DOCTYPE html>
html>
head>
title>Welcome to nginx!</title>
style>
tml { color-scheme: light dark; }
ody { width: 35em; margin: 0 auto;
ont-family: Tahoma, Verdana, Arial, sans-serif; }
/style>
/head>
body>
h1>Welcome to nginx!</h1>
p>If you see this page, nginx is successfully installed and working.
urther configuration is required for the web server, reverse proxy,
PI gateway, load balancer, content cache, or other features.</p>

p>For online documentation and support please refer to
a href="https://nginx.org/">nginx.org</a>.<br/>
o engage with the community please visit
a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
or enterprise grade support, professional services, additional
ecurity features and capabilities please refer to
a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

p><em>Thank you for using nginx.</em></p>
/body>
/html>
ll commands and output from this session will be recorded in container logs, including credentials and sensitive information passed thro
f you don't see a command prompt, try pressing enter.
arning: couldn't attach to pod/test-pod, falling back to streaming logs: Internal error occurred: Internal error occurred: error attachi
TAINER_EXITED state
!DOCTYPE html>
html>
head>
title>Welcome to nginx!</title>
style>
tml { color-scheme: light dark; }
ody { width: 35em; margin: 0 auto;
ont-family: Tahoma, Verdana, Arial, sans-serif; }
/style>
/head>
body>
h1>Welcome to nginx!</h1>
```

10. Pod with Resource Limits and Requests Task: Add resource requests and limits to the containers in your YAML file. Specify CPU and memory requests/limits for both containers and deploy the Pod. Use kubectl describe pod to verify if the resource configurations are correctly applied.

Objective:

Configure CPU and memory resource requests and limits for both containers inside a Pod.

Implementation Steps:

1. Create a YAML file.

   vi resource-pod.yml

2. Add the following configuration:

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
    - name: nginx1
      image: nginx
      resources:
        requests:
          cpu: "100m"
          memory: "128Mi"
        limits:
          cpu: "500m"
          memory: "256Mi"

    - name: nginx2
      image: nginx
```

```
    resources:
      requests:
        cpu: "200m"
        memory: "128Mi"
      limits:
        cpu: "500m"
        memory: "256Mi"
```

3. Save and exit the file.
4. Deploy the Pod.

   kubectl apply -f resource-pod.yml
5. Verify the Pod status.

   kubectl get pods
6. Check whether both containers are running.

   kubectl get pod resource-pod

   The READY column should display 2/2.
7. Describe the Pod.

   kubectl describe pod resource-pod
8. Verify the CPU and memory requests under the container resource configuration.
9. Verify the CPU and memory limits for both containers.
10. Check the current resource utilization.

    kubectl top pod resource-pod --containers

    Expected Result: Both containers should be created successfully with their respective CPU and memory requests and limits.

root@master: ~

```
root@master:~# nano resourcepod.yml
root@master:~# cat resourcepod.yml

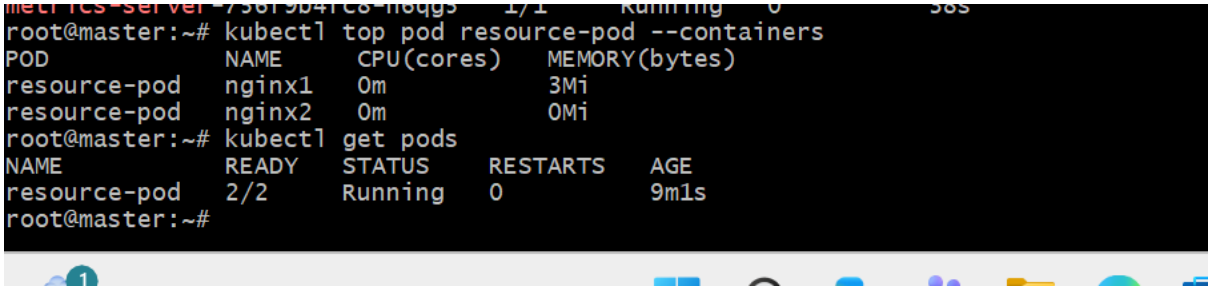
apiVersion: v1
kind: Pod
metadata:
  name: resource-pod
spec:
  containers:
    - name: nginx1
      image: nginx
      resources:
        requests:
          cpu: "100m"
          memory: "128Mi"
        limits:
          cpu: "500m"
          memory: "256Mi"

    - name: nginx2
      image: busybox
      command: ["sh", "-c", "sleep 3600"]
      resources:
        requests:
          cpu: "100m"
          memory: "128Mi"
        limits:
          cpu: "500m"
          memory: "256Mi"

root@master:~# kubectl apply -f resourcepod.yml
pod/resource-pod created
root@master:~# kubectl get pods
NAME           READY   STATUS    RESTARTS   AGE
resource-pod   2/2     Running   0          9s
root@master:~# kubectl describe pod resource-pod
Name:             resource-pod
Namespace:        default
Priority:         0
Service Account:  default
Node:             worker02/172.31.11.52
Start Time:       Fri, 06 Nov 2026 13:37:56 +0000
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.2.7
IPs:
  IP:  10.244.2.7
```

root@master: ~

```
    Command:
      sh
      -c
      sleep 3600
    State:          Running
      Started:      Fri, 06 Nov 2026 13:37:57 +0000
    Ready:          True
    Restart Count:  0
    Limits:
      cpu:     500m
      memory:  256Mi
    Requests:
      cpu:        100m
      memory:     128Mi
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-5h8wd (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-5h8wd:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   Burstable
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  24s   default-scheduler  Successfully assigned default/resource-pod to worker02
  Normal  Pulling    24s   kubelet            spec.containers{nginx1}: Pulling image "nginx"
  Normal  Pulled     23s   kubelet            spec.containers{nginx1}: Successfully pulled image "nginx" in 146ms (146ms incl
  Normal  Created    23s   kubelet            spec.containers{nginx1}: Container created
  Normal  Started    23s   kubelet            spec.containers{nginx1}: Container started
  Normal  Pulling    23s   kubelet            spec.containers{nginx2}: Pulling image "busybox"
  Normal  Pulled     23s   kubelet            spec.containers{nginx2}: Successfully pulled image "busybox" in 476ms (476ms in
  Normal  Created    23s   kubelet            spec.containers{nginx2}: Container created
  Normal  Started    23s   kubelet            spec.containers{nginx2}: Container started
root@master:~#
```


root@master:~# kubectl top pod resource-pod --containers
POD NAME CPU(cores) MEMORY(bytes)
resource-pod nginx1 0m 3Mi
resource-pod nginx2 0m 0Mi
root@master:~# kubectl get pods
NAME READY STATUS RESTARTS AGE
resource-pod 2/2 Running 0 9m1s
root@master:~#